

PART 9 OF 18

Tool Governance for Enterprise AI

Tool Registry, Supply Chain Security, Runtime Validation, Secrets Management, MCP Governance, Tool Trust Scoring, and Lifecycle Management

ENTERPRISE AI CONTROL ARCHITECTURE

Implementation Guide for Production AI Systems • 2026

9.1 The Tool Governance Imperative

Tools are the primary interface through which AI agents affect the world. A tool invocation can read sensitive data, modify production systems, make financial transactions, communicate on behalf of the enterprise, or trigger downstream automated processes. The security posture of an agent deployment is fundamentally bounded by the security posture of its tools: a perfectly aligned agent with poorly governed tools is a serious security risk.

■

Tool Governance Scope: Includes all mechanisms by which agents invoke external functionality—MCP servers, OpenAPI-defined REST endpoints, function calling, subprocess execution, database queries, file system operations, SDK method calls, and internal microservice calls via any protocol.

9.2 Enterprise Tool Registry

9.2.1 Registry Architecture

Every tool available to enterprise AI agents must be registered in a centralized Tool Registry before any agent can invoke it. The registry serves as the single source of truth for tool metadata, trust assessments, capability definitions, and version history. Agents cannot discover or invoke unregistered tools.

Registry Field	Description	Source
Tool ID	Unique, immutable identifier (UUID)	Registry on registration
Tool Name + Version	Human-readable name and semantic version	Tool developer on submission
Tool Type	MCP / REST / Function / Internal / External	Tool developer on submission
Capability Spec	Structured definition of what the tool can do (read/write/execute/external)	Tool developer on submission
Endpoint / Package	URL, NPM package, or deployment reference	Tool developer on submission
Trust Score	0-100 computed trust score	Registry computed on approval
Security Assessment	Last security review findings and date	Security team
SBOM	Software Bill of Materials for tool implementation	Tool developer on submission
Allowable Agents	Agent types authorized to use this tool	AI Security team
Rate Limits	Maximum invocations per agent per time period	Ops team configuration
Data Classification	Classification of data the tool can access	Data governance team
Approval Status	Draft / Review / Approved / Deprecated / Retired	Registry workflow

Registry Field	Description	Source
Behavioral Tests	Last behavioral test run results and date	Automated testing pipeline

9.3 Tool Approval Workflow

9.3.1 Tool Submission and Review Process

1. Developer Submission

Tool developer submits tool specification, SBOM, source code reference, and self-assessment. Tool receives DRAFT status.

2. Automated Scanning

CI/CD pipeline scans tool code for: known vulnerabilities (CVE matching), secrets in code, dependency vulnerabilities, license compliance. Tools with critical findings blocked at this stage.

3. Security Review

AI Security team reviews: tool capability scope (does spec match implementation?), data access scope, external call patterns, secret handling, error handling (does tool expose sensitive data in errors?).

4. Behavioral Testing

Tool deployed in isolated sandbox with monitoring. Automated test suite runs: normal operation tests, adversarial input tests, boundary condition tests, injection attempt tests. Behavioral baseline captured.

5. Privacy Review

Data governance team reviews: what data the tool accesses, whether it sends data externally, retention of data within tool, GDPR/CCPA compliance for user data.

6. Approval Decision

Joint decision by AI Security + Data Governance + AI Platform teams. Approval may be conditional (limited to specific agent types or data classifications).

7. Registry Publication

Tool published to registry with metadata, trust score, and allowable agent list. Version pinned.

9.4 MCP (Model Context Protocol) Governance

9.4.1 MCP Security Model Gaps

MCP, while providing a standardized interface for tool use, has significant security gaps in its base specification that enterprises must address through additional controls:

Gap	Risk	Mitigation
No authentication in base spec	MCP client-server communication has no mandatory authentication; any client can call any server	Implement mTLS for all MCP connections; MCP server validates client certificate against agent allowlist
No response integrity	MCP tool results are not signed; MITM can modify results without detection	Implement HMAC signatures on all MCP responses; client verifies signature before processing
No server discovery security	MCP server discovery (stdio, SSE) lacks integrity guarantees	Use registry-controlled server URLs; reject dynamic discovery from untrusted sources
No audit trail in spec	MCP does not mandate logging of tool calls or results	Implement audit proxy that logs all MCP traffic including full request/response
No rate limiting	MCP does not define rate limiting; DoS possible through tool calls	Implement rate limiting at the MCP proxy layer
Tool result injection	Malicious MCP server can return results containing injection payloads	Scan all MCP tool results for injection patterns before including in agent context

9.5 Tool Trust Scoring

9.5.1 Trust Score Components

Tool trust scores enable dynamic, risk-proportionate access decisions. A higher-trust tool can be invoked more freely; lower-trust tools require additional approval. Trust scores are computed from multiple factors and updated continuously.

Factor	Weight	Score Components
Provenance	20%	Open source (verifiable) vs closed source; vendor reputation; CVE history of developer
Code Quality	15%	Static analysis findings; test coverage; dependency vulnerability count
Behavioral Testing	25%	Pass rate on security behavioral tests; anomaly rate in sandbox testing; injection resistance score
Production History	20%	Invocation volume; error rate; incident history; time since last security issue
External Assessment	10%	Third-party security audit; CVE disclosures; vendor security certifications
Data Handling	10%	Scope of data access; external data transmission; retention practices

9.5.2 Trust Score Usage in Authorization

Trust Score	Label	Authorization Policy
90-100	Highly Trusted	Agent can invoke without approval; standard rate limits apply
70-89	Trusted	Agent can invoke with enhanced logging; rate limits enforced
50-69	Conditionally Trusted	Invocation permitted only for approved task types; elevated logging
30-49	Low Trust	Human approval required for each invocation in sensitive contexts
10-29	Restricted	Human approval required for all invocations; supervisor AI monitor required
0-9	Untrusted	Not invocable by agents; requires re-assessment before use

9.6 Secrets Management for Tool Credentials

Tools require credentials to access external services—API keys, OAuth tokens, database passwords, and certificates. These secrets must never be embedded in agent prompts, hardcoded in tool configurations, or accessible through agent memory. A dedicated secrets management system must provide dynamic, short-lived credentials to tools at invocation time.

9.6.1 Secrets Architecture

- **HashiCorp Vault / AWS Secrets Manager / Azure Key Vault:** Enterprise secrets vault; tools retrieve credentials via authenticated API at runtime
- **Dynamic Secrets:** Credentials generated on-demand per tool invocation with minimal TTL (e.g., 5-minute database passwords)
- **Secrets Injection:** Secrets injected into tool execution environment at invocation time via environment variables or volume mounts; never persisted
- **Credential Rotation:** All credentials rotated on defined schedule; breached credentials invalidated within 15 minutes
- **Agent Secret Isolation:** Each agent instance has access only to secrets for its authorized tool set; no cross-agent secret sharing
- **Audit of Secret Access:** Every secret access logged with: which agent, which tool, which task, timestamp; anomalous access triggers alert

9.7 Tool Observability

Every tool invocation must generate telemetry data that enables: performance monitoring, security audit, behavioral analysis, and incident investigation. Tool observability is a first-class architectural requirement, not an afterthought.

Telemetry Type	Collected Data	Retention	Primary Use
Invocation Log	Tool ID, agent ID, task ID, parameters (masked), timestamp, duration	90 days	Audit, debugging
Result Log	Tool ID, result type, result classification, size, anomaly flags	90 days	Security analysis
Error Log	Tool ID, error type, error message (sanitized), retry count	30 days	Operations
Performance Metric	Latency, error rate, timeout rate per tool and agent type	1 year	Operations, SLA
Behavioral Metric	Usage patterns, parameter distributions, result patterns	1 year	Threat detection
Security Event	Policy violations, anomaly detections, injection attempts	3 years	Compliance, forensics